M1 — Literature survey, agent design and project plan

Name: Indrakshee Mukherjee **Team:** A-Team **Specialisation:** Gate - Player (Strategy)

1 Literature survey

My literature review served the goal of understanding where exactly it would be beneficial to have Large Language Model (LLM) calls and where it would be more suitable to have a more direct analytical or algorithmic approach.

1.1 Determining Unsolvability

Previous research has shown work on developing algorithms that can be used to detect if a given problem is solvable or not. Moreover it runs the algorithm at different points of solving the puzzle so that we are able to tell as early as possible if we should even begin attempting the puzzle [1]. This is a useful strategy because skipping puzzles that cannot be solved would allow us to save tokens and not waste resources on a problem unnecessarily.

1.2 Decision Accuracy

It might not always be possible to decide accurately if a given puzzle can be solved or not but we must strive to do this as well as possible. LLMs are also not known for always being able to decide if a specific problem can be solved or not, it would not make sense to outsource this decision to an LLM. However, since an LLM always needs to return an answer, even if it does not have the right answer or cannot reason it, it hallucinates and makes something up [2]. This cannot necessarily be verified or verified accurately during the game as we use the answer it has delivered to continue with the steps meaning that we might be unnecessarily continuing with an approach that does not work because the LLM misguided the agent. Using the Tokens per Correctness (TPC) [2] metric, it can be evaluated how accurately an LLM has decided which approach to take or decided if a given puzzle is solvable or not, which would allow us to in turn use fewer tokens to complete the quest.

2 Agent design

2.1 My agent

The problem: The player's main task is to solve the puzzles that are in the setlist and skip puzzles accordingly. Since there is neither a fixed rule as to when puzzles can or should be skipped nor a limit as to how many puzzles can be skipped, it is important to decide correctly which ones should indeed not be attempted. I devised the following steps that my agent would execute in order. There was a need to reduce dependency on LLM calls as it would be difficult to verify if the LLM has indeed made the right decision or not [3]. This is also due to the fact that we only have access to one LLM and using the strategy of an LLM council to cross-check the decision is not applicable here. In some cases, we also do not require further text to be generated, making an LLM call fairly redundant. Moreover, we need to reduce token usage and that requires as few LLM calls as possible so a quest is better rewarded when it is able to

successfully solve a puzzle with minimal LLM usage. If a puzzle could have been solved without an LLM call, it scores better.

1. Call on a classification model to classify the puzzle
2. Route to a decision model to determine if the puzzle should be skipped or solved
3. Call on the appropriate algorithm if puzzle is to be solved or skip

My Approach: Using classical Natural Language Processing (NLP) techniques, the first step would be to parse and analyze the text of the puzzle to understand what sort of puzzle we are dealing with. This step would ensure that we call on the correct tool because if the puzzle is a mathematical one, then depending on the complexity of the mathematics, it might just be easiest to call on a calculator. This is guaranteed to give us the right answer in comparison with an LLM that often makes mistakes even when it comes to simple arithmetic. Of course, it is not expected that every mathematical problem can be solved directly by a calculator but given enough clues and the state that we are in, we are able to use text to feature models to potentially solve the puzzle. So in the first step, statistical methods like k-means clustering and decision trees can also be relied upon to determine what type of puzzle is currently at hand [4].

Once it has been classified what sort of puzzle it is, it must then be decided if the puzzle is even solvable or not. This stage is crucial, because the agent should not even begin to solve the puzzle if it cannot be solved. However, it is also possible that this decision is not the most accurate and we might end up trying to solve a puzzle that we cannot solve or not attempting a puzzle that we skipped because it was wrongly classified as unsolvable. The goal would be to have as high precision as possible so that we are not constantly classifying and then deciding on whether to skip or solve the puzzle wrongly.

There are many known puzzle types and also solvers for those puzzles. Many of these are programmable and can be function calls that the agent then deploys. However, if the puzzle type is unknown or the function does not exist, it would need to fall back on an LLM to solve the puzzle. At this point, NLP techniques come in handy again as the prompt to call the LLM would need to properly describe what the puzzle is to ensure that the LLM is able to solve it properly or at the latest at this juncture advise to skip the problem. If necessary, the LLM can also call on the appropriate algorithm to solve the puzzle.

Where it meets the team: The player enables shortcuts to be found upon successfully solving a puzzle so this essentially allows the improviser to find a better route knowing that it has certain shortcuts in the maze.

2.2 The Ensemble

Dependencies. The parts depend on each other in the four ways named by Malone and Crowston [5]. There are two *shared resources*: the places on the trolley and the one connection to the arena. As *producer and consumer*, the Improviser delivers the volumes that the Musicologist can read only in the hall, and a door opened by the Player gives the Improviser a shortcut. *Simultaneity* is needed at a door, where a puzzle can be answered only while the trolley stands there [6]. Each question is a *task with subtasks*: choosing rooms, fetching, reading and answering.
Coordination. The Ensemble is organised as a hierarchy [7]. At the top is the Conductor, written in plain code. It holds the connection, runs the loop, checks every action and hands each decision to one specialist (Fig. 1). It also keeps the shared store: the setlist texts, sent only once, the map, the texts read and the costs. In a base quest, the first message names the quest and so the specialist. In the Overture the order of work is open, so a fixed rule settles where the

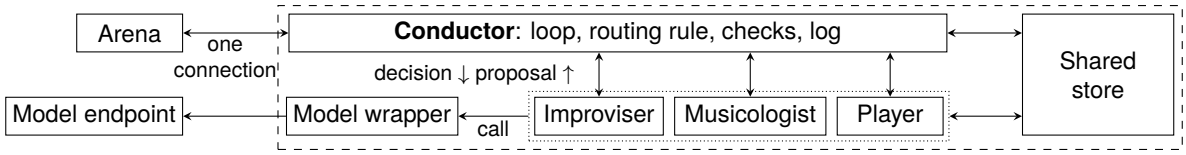


Figure 1: The container (dashed) with its three specialists (dotted). The Conductor and the specialists use the shared store. Only the specialists call the model, through the wrapper.

trolley goes next: the Musicologist rates each candidate sign or volume, the Improviser gives its distance, and the Conductor takes the candidate with the most worth per distance.

The alternative considered was a blackboard [8], on which a control component selects which part writes next [7]. Its authors noted that such control "takes time and requires many fairly sophisticated mechanisms" and is justified where no explicit algorithm exists [8, p. 246]. Here a simple rule chooses the next step without using tokens, so only the shared store was taken over.

When a part fails, is slow or is wrong. If a specialist fails, the Conductor falls back on a default without tokens. For a question, it sends the best answer so far, which the Musicologist keeps up to date in the shared store. A puzzle is skipped, and on the map the nearest exit is taken. A slow model call is stopped before the endpoint's 300-second cut-off [9] and retried once, so a turn stays below the ten minutes of silence that end a run [6]. Against wrong answers, every proposal is first checked in code, as language models were found no better at verifying solutions than at producing them [10]. A failed proposal is returned once with the reason. If it fails again, the last well-formed one is sent, as a wrong answer costs no more than a skip [6], except on claims, which a team of three does not get.

Logging. The starter code logs every arena message and model call [9]. For each action, the log will add the deciding part, its reason, its tokens, the change in `spent` and the check result.

3 Project plan

3.1 The team's plan

By	What will be working	Who
16 Nov	Self-test passes, and fails when the prompt is broken	All
20 Nov	Conductor shares: loop and routing (Jurcik), shared store (Merkl), logging and failure rules (Mukherjee)	one each
23 Nov	Each base quest plays a practice run via the Conductor	each owner
27 Nov	**First Overture run with all parts in one container**	All
30 Nov week	Rehearsal: image connects to course arena and endpoint	Jurcik
4 Dec	Overture end to end with failure rules, baselines measured	Mukherjee
8 Dec	Image frozen, all dependencies inside, reflections drafted	Merkl
10 Dec	M2 handed in	each

Each member owns one Conductor share. Open rows are checked weekly and work is moved.

3.2 My plan

Experiment with various HuggingFace models to ensure classification and decision steps are optimized followed by the right Python libraries being called for puzzle solving and ultimately relying on LLM calls. Check that puzzles that can be solved are solved and the rest are skipped.

References

[1] S. Mukhopadhyay, A. Qidwai, J. Sen, and M. Shrivastava, "RUBICON: Do language models know when they've crossed the point of no return?" In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, H.-Y. Lee, S. Dev, J. J. Li, and D. Elliott, Eds., Budapest, Hungary: Association for Computational Linguistics, Oct. 2026, pp. 61 499–61 530. DOI: 10.18653/2026.emnlp-main.2704

[2] R. Xie et al., *Over-searching in search-augmented large language models*, 2026. arXiv: 2601.05503 [cs.LG].

[3] A. R. Gosthipaty, L. Debut, P. Cuenca, and T. Aarsen, *Decision models in hugging face*, https://huggingface.co/blog/decision, Nov. 2026.

[4] MathWorks, *Choosing the best machine learning classification model and avoiding overfitting*, https://de.mathworks.com/campaigns/offers/next/choosing-the-best-machine-learning-classification-model-and-avoiding-overfitting.html, Accessed: 2026-11-12, n.d.

[5] T. W. Malone and K. Crowston, "The interdisciplinary study of coordination," *ACM Computing Surveys*, vol. 26, no. 1, pp. 87–119, 1994. DOI: 10.1145/174666.174668

[6] IN2106 Multi-Agent AI Lab, *The Archive Games: Rules and API reference*, Course document, IN2106 Multi-Agent AI Lab, Technical University of Munich, Nov. 9, 2026.

[7] J. Schaumeier, *How do several agents decide together?* Lecture slides, Session 2, IN2106 Multi-Agent AI Lab, Technical University of Munich, Oct. 26, 2026.

[8] L. D. Erman, F. Hayes-Roth, V. R. Lesser, and D. R. Reddy, "The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty," *ACM Computing Surveys*, vol. 12, no. 2, pp. 213–253, 1980. DOI: 10.1145/356810.356816

[9] J. Schaumeier, *The Archive Games*, Lecture slides, Session 4, IN2106 Multi-Agent AI Lab, Technical University of Munich, Nov. 9, 2026.

[10] S. Kambhampati et al., "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, PMLR, 2024, pp. 22 895–22 907. arXiv: 2402.01817.

Use of AI

Gemini 3.1 Pro supported the literature survey, citations and deliberation of the chosen approach. There was no further use of AI.